KUBERNETES COMPREHENSIVE · AXP-K8S-5D

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS · Minikube
Built	11 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 2 — Reliability, Resource Governance and Controlled Change

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 2

What changed today

Yesterday you built a platform. Today you made it trustworthy.

Yesterday	Today
Pods land wherever the scheduler guesses	Every workload declares what it needs
`Running` is the only health signal	Three probes, separated by consequence
Capacity is whatever you typed	Autoscaling on real CPU load
Only Deployments	Deployment, DaemonSet, Job, CronJob
A deploy drops payments	Zero-downtime release, measured

Chapter structure is the same 16-point template as Chapter 1.

2.1 Resource requests and limits

1. What it is

Two numbers per container. **Requests** tell the scheduler how much to reserve. **Limits** tell the kernel what ceiling to enforce at runtime. They feed two different systems that never talk to each other.

2. Why it exists

Without requests, the scheduler has no idea what a pod needs and packs nodes until they collapse. Without limits, one leaking process takes the whole node down — including every unrelated workload on it.

3. The business problem

AxisPay's p99 authorisation is **217 ms** against a **300 ms** SLO. That is 83 ms of headroom.

A merchant reports intermittent slowness. Nothing is down. Nothing has restarted. The logs are clean. The cause is a CPU limit set by someone who wanted to be careful, and it is consuming the entire 83 ms — with no log line, no event and no restart.

4. How it works

```
SCHEDULING                          RUNTIME
uses requests only                  uses limits only

node allocatable   2000m            CPU over limit  -> THROTTLED
already reserved   1400m            MEM over limit  -> OOMKILLED (137)
pod requests        300m -> fits
                   -> 300m RESERVED
```

The reservation is not a measurement. A node that is 5% busy but 95% *reserved* will refuse new pods. This surprises people the first time they see an idle node reject work.

5. Internal architecture

The kubelet translates requests and limits into Linux cgroup v2 settings:

Field	cgroup control	Effect
`requests.cpu`	`cpu.weight`	Relative share when the CPU is contended
`limits.cpu`	`cpu.max` (quota/period)	Hard throttle — the process is descheduled for the rest of the period
`requests.memory`	*(scheduling only)*	Not enforced at runtime
`limits.memory`	`memory.max`	Exceed it and the kernel OOM-killer terminates the process

CPU quota is enforced per **100 ms period**. A container limited to 500m gets 50 ms of CPU per 100 ms. Burn it in 20 ms and you are frozen for the remaining 80 ms — which is where sawtooth latency comes from.

6. Component interactions

```
you             declare requests + limits
scheduler       filters nodes by unreserved capacity (requests)
kubelet         writes cgroup values (limits)
kernel          throttles CPU / OOM-kills on memory
metrics-server  scrapes actual usage
HPA             usage ÷ REQUEST = utilisation
```

Requests appear twice: once as the scheduler's input, once as the HPA's denominator. That is why one wrong number breaks two systems.

7. Enterprise example

A payments platform runs a quarterly "right-sizing" review: p95 usage over 30 days becomes the new request, p99 plus 100% becomes the limit. Requests that are too high are as expensive as limits that are too low — over-requesting silently makes cluster capacity unusable for everyone else, and it does not appear in your own metrics at all.

8. Real-world analogy

A restaurant booking. The **request** is the table you reserve — held for you whether or not you turn up. The **limit** is the maximum party size that table will take. Reserve a table for two and try to seat six, and you are turned away.

Where it breaks: a restaurant cannot seat a walk-in at your reserved table. Kubernetes *can* let another pod burst into CPU you reserved but are not using. Reserved CPU is not idle CPU.

9. Best practices

Practice	Reason
Measure before you set	`kubectl top` under load, several samples, not one idle reading
`request` = steady state + ~30%	The scheduler must reserve for a normal day
`limit` = peak + ~100%	Room for a genuine spike without eating the node
Always set a memory limit	Without one, a leak takes down the node, not just the pod
Be cautious with CPU limits	Throttling is invisible. Many teams set memory limits and omit CPU limits deliberately.
Never set `limit` < `request`	Nonsensical: the scheduler reserves more than the kernel allows. Always a defect.
Re-measure after every significant release	v1.1.0 added two HTTP client pools to `payment-service`

10. Common mistakes

Mistake	Symptom
No requests at all	BestEffort QoS, evicted first, HPA reports `<unknown>`
CPU limit too low	Latency breach with **no log, no event, no restart**
Memory limit too low	`OOMKilled`, exit 137, CrashLoopBackOff
`limit` below `request`	Pod schedules then dies immediately — this is INC-2
Requests set far too high	Cluster capacity silently wasted; invisible in your own metrics
Setting limits == requests everywhere	Guaranteed QoS, zero burst headroom, throttled at exactly the wrong moment
Sizing from one idle sample	Correct at 03:00, throttled at 09:00

11. Security considerations

Threat	Control	Residual risk
One tenant exhausts a node (noisy neighbour / DoS)	Limits on every container; ResourceQuota per namespace	A pod can still saturate network or disk I/O
Memory-exhaustion attack on a node	Memory limits + eviction thresholds	Kernel-level exhaustion still possible
Pod with no limits admitted	LimitRange `default`	Only applies to pods created after it
Resource-based side channels between tenants	Node isolation for sensitive workloads	Shared kernel remains shared

12. Performance considerations

- **CPU throttling is invisible.** The only signals are latency and `nr_throttled` / `throttled_usec` in `/sys/fs/cgroup/cpu.stat`.
- **The 100 ms period matters.** A latency-sensitive service can be badly throttled while showing modest average CPU, because the burst is concentrated.
- **Memory has no throttle.** There is no gentle degradation — only OOMKill.
- **Over-requesting costs real money.** Reserved-but-unused capacity cannot be scheduled by anyone.
- **Leading indicator:** `container_cpu_cfs_throttled_periods_total / container_cpu_cfs_periods_total` above ~5% means the limit is too low.

13. High availability

Resources are a per-pod concern, but they determine cluster-level survivability. Requests that are too low let the scheduler overcommit a node; when reality catches up, the kubelet evicts — **BestEffort first, then Burstable, Guaranteed last**. Accurate requests are what make eviction ordering meaningful.

14. Disaster recovery

Resource settings are configuration and recover with the manifest. The failure worth planning for is **node pressure**: if a node runs out of memory, the kubelet evicts pods by QoS class and by how far each exceeds its request. Workloads that must survive should have accurate requests and, for the truly critical, a `PriorityClass` (Day 4).

15. Monitoring

Metric	Threshold
`container_cpu_cfs_throttled_periods_total` ratio	> 5% of periods
`container_memory_working_set_bytes / limit`	> 85%
`kube_pod_container_status_last_terminated_reason{reason="OOMKilled"}`	Any — **page**
`kube_pod_container_resource_requests` vs actual usage	Persistent gap = over-requesting
`kube_resourcequota` used vs hard	> 80%

16. Troubleshooting

Symptom	Cause	Command	Fix
Slow, no errors, no restarts	**CPU throttling**	`kubectl exec … -- cat /sys/fs/cgroup/cpu.stat`	Raise the CPU limit
`OOMKilled`, exit 137	Memory limit too low, or a leak	`kubectl describe pod` → Last State	Raise the limit; check for a leak
`Pending`, "Insufficient cpu"	No node has enough **unreserved**	`kubectl describe pod`; `kubectl describe node`	Lower requests or add capacity
`exceeded quota`	Namespace budget full	`kubectl describe quota -n <ns>`	Lower requests or raise the quota
Pod dies seconds after starting, clean logs	`limit` below what the process needs	`kubectl describe pod` → Last State	Raise the limit — this is INC-2
Node evicting pods	Node under memory pressure	`kubectl describe node` → Conditions	Fix requests; add capacity

Interview questions

1. **Which does the scheduler use — requests or limits?** *Requests only. Limits are enforced by the kernel at runtime and are invisible to scheduling.*
2. **What happens when a container exceeds its CPU limit? Its memory limit?** *CPU: throttled — slowed down, no error, no event, no log. Memory: OOMKilled, exit 137, restarted. CPU is compressible, memory is not.*
3. **What are the three QoS classes and how are they assigned?** *Guaranteed (requests == limits for every container and both resources), Burstable (requests set, limits higher or absent), BestEffort (nothing declared). Eviction order is BestEffort, Burstable, Guaranteed.*

4. **Why might you deliberately NOT set a CPU limit?** *(senior) Because throttling is invisible and often worse than the problem it prevents. Requests already guarantee a share under contention. Many teams set memory limits (to contain leaks) and omit CPU limits (to allow bursting), accepting noisy-neighbour risk in exchange for predictable latency. It is a real trade with real advocates on both sides.*
5. **A pod has `requests.memory: 96Mi` and `limits.memory: 48Mi`. What happens and what should have caught it?** *(senior) The scheduler reserves 96Mi; the kernel kills the process above 48Mi. It schedules and then dies immediately, repeatedly. A validating admission policy or a CI policy check should reject any manifest where a limit is below its request — it is never correct.*

2.2 ResourceQuota and LimitRange

1. What it is

Namespace-level governance. A **ResourceQuota** caps the total a namespace may consume. A **LimitRange** supplies per-container defaults and enforces minimum, maximum and ratio bounds.

2. Why it exists

Per-pod resources are the developer's concern. Namespace ceilings are the platform team's. Without them, any namespace can consume the whole cluster.

3. The business problem

A misconfigured CI pipeline created 400 pods in a staging namespace on a shared cluster. It exhausted the node pool, and **production payment pods could not be scheduled for eleven minutes.** No malice, no application bug — just no ceiling.

4. How it works

Both are **admission-time** controls. They run when an object is created and reject it if it does not fit. Neither affects anything already running.

```
Deployment submitted
   -> LimitRange     fills in missing requests/limits, checks min/max/ratio
   -> ResourceQuota  is there room in the namespace budget?
   -> admitted, or REJECTED (the pod is never created)
```

5. Internal architecture

The dependency between them catches everyone. A ResourceQuota on `requests.cpu` makes any pod *without* a CPU request unschedulable — Kubernetes cannot count what was never declared. The LimitRange's `defaultRequest` supplies the missing number.

A ResourceQuota without a LimitRange breaks every manifest that forgot to set resources.

AxisPay's quota on `axispay-core`:

Resource	Hard	Sized for
`requests.cpu`	2	1920m at full HPA scale
`requests.memory`	2Gi	1520Mi at full HPA scale
`limits.cpu`	9	8700m at full HPA scale
`limits.memory`	5Gi	3904Mi at full HPA scale
`pods`	40	~19 at peak

> **Requests must fit real capacity. Limits may be oversubscribed.** The scheduler reserves requests, so their sum can never exceed what the cluster has. Limits are a per-container ceiling; it is normal and correct for their sum to exceed capacity — that is what Burstable QoS is for.
>
> But the quota must still allow the HPA to reach `maxReplicas`, or autoscaling stops partway up with `FailedCreate` on the ReplicaSet — during exactly the spike it exists to absorb.

6. Component interactions

The rejection surfaces on the **ReplicaSet**, not the Deployment. The Deployment reports `0/1` and looks fine. `kubectl describe rs` is where the `FailedCreate` event lives. This is the most common way a quota rejection gets misdiagnosed.

7. Enterprise example

A bank automates namespace creation: requesting `payments-prod` provisions a ResourceQuota sized to the team's budget, a LimitRange with sensible defaults, a default-deny NetworkPolicy, RBAC bound to the owning group, and Pod Security labels. Nobody creates a namespace by hand, so nobody forgets a control.

8. Real-world analogy

A departmental budget with a per-item spending cap. The ResourceQuota is the annual budget; the LimitRange is "no single purchase over R50,000, and nothing under R100 needs approval". You can be refused for exceeding either.

9. Best practices

Practice	Reason
Never a quota without a LimitRange	Otherwise every under-declared manifest is rejected
Size the quota for HPA max, not steady state	Or autoscaling fails during the spike
Quota object counts, not just compute	Stops a runaway controller creating 10,000 objects
Set `maxLimitRequestRatio`	Stops workloads lying to the scheduler
Automate namespace creation with its controls attached	Manual creation always forgets something
Alert at 80% quota utilisation	Before rejections start, not after

10. Common mistakes

Mistake	Symptom
Quota without LimitRange	Every bare manifest rejected; confusing for developers
Quota sized for steady state	HPA silently stops mid-scale during a spike
Only checking `kubectl get deploy`	The rejection is on the ReplicaSet; you never see it
Forgetting object-count quotas	A CI loop fills etcd with Jobs
Quota on limits but not requests	The scheduler can still overcommit the nodes

11. Security considerations

Threat	Control	Residual risk
Resource-exhaustion DoS by one tenant	ResourceQuota per namespace	Network and disk I/O are not quota'd
Object-count flooding of etcd	`count/*` quotas	Cluster-scoped objects are not covered
Privileged over-provisioning	RBAC on quota objects themselves	Cluster-admin can always raise them

12. Performance considerations

Quota evaluation is a synchronous admission step and adds a small latency to every create. At very high object-creation rates the quota controller can become a bottleneck — visible as `apiserver_admission_controller_admission_duration_seconds` climbing for `ResourceQuota`.

13. High availability

Governance objects are metadata and inherit control-plane HA. Their contribution to availability is indirect but real: they prevent one namespace from making the cluster unschedulable for everyone else.

14. Disaster recovery

Trivially reproducible from manifests. The one operational care: raising a quota during an incident is a legitimate break-glass action, but it must be reflected back into Git or the next apply reverts it — quietly, at the worst possible time.

15. Monitoring

Metric	Threshold
`kube_resourcequota{type="used"} / {type="hard"}`	> 80%
`FailedCreate` events on ReplicaSets	Any
`apiserver_admission_controller_admission_duration_seconds{name="ResourceQuota"}`	Rising

16. Troubleshooting

Symptom	Cause	Command	Fix
`exceeded quota`	Budget full	`kubectl describe quota -n <ns>`	Lower requests, scale down, or raise the quota
Deployment `0/1`, no pod	Quota rejection on the ReplicaSet	`kubectl describe rs -n <ns>`	Read the FailedCreate
Every new pod rejected	Quota with no LimitRange	`kubectl get limitrange -n <ns>`	Add a LimitRange with `defaultRequest`
`maximum cpu usage per Container`	Exceeds LimitRange `max`	`kubectl describe limitrange`	Lower the limit
`ratio … is higher than max`	request/limit gap too wide	Same	Raise the request or lower the limit
HPA stalls below max	Quota reached mid-scale	`describe hpa`; `describe quota`	Size the quota for HPA max

Interview questions

1. **Why does a ResourceQuota need a LimitRange?** *A quota on requests makes pods without requests unschedulable — the quota cannot count an undeclared value. The LimitRange's `defaultRequest` supplies it. Without one, a quota breaks every manifest that omitted resources.*
2. **Where does a quota rejection appear?** *On the ReplicaSet, as a `FailedCreate` event. The Deployment just reports fewer replicas than desired. Checking only `kubectl get deploy` hides the cause completely.*
3. **Why can `limits` be oversubscribed but `requests` cannot?** *(senior) Requests are reservations the scheduler must honour, so their sum is bounded by real capacity. Limits are per-container ceilings enforced by the kernel; a pod may burst into headroom neighbours are not using. Oversubscribing limits is how Burstable QoS delivers value.*
4. **How do you size a quota for a namespace with autoscaling?** *(senior) For `maxReplicas`, not steady state. A quota sized to normal load causes the HPA to stop scaling partway up, with `FailedCreate` on the ReplicaSet — during the exact traffic spike it exists to absorb, and with a symptom that looks nothing like the cause.*

2.3 Health probes

1. What it is

Three checks the kubelet runs against your container, each with a **different consequence** when it fails.

2. Why it exists

Without probes, the only health signal Kubernetes has is "did the process start?" — which is not the same as "can this pod take a payment?"

3. The business problem

Yesterday's incident took out **all three** `payment-service` replicas simultaneously. A bad image rolled out to every pod and nothing stopped it. The review asked: *why did the rollout continue after the first pod failed?*

Because Kubernetes had no way to know it had failed.

4. How it works

Probe	Question	Consequence of failure	Check dependencies?
startup	Has initialisation finished?	Keep waiting; **liveness suspended**	No
liveness	Is this process unrecoverable?	**Container RESTARTED**	**Never**
readiness	Can *this pod* serve right now?	**Removed from endpoints.** Not restarted.	**Yes — that is the point**

> **Learn the consequence, not the name.** Every probe mistake in production is made by someone who knew the definitions.

AxisPay's timings:

Probe	period × threshold	Reacts in
startup	2 s × 30	60 s budget to start
liveness	10 s × 3	30 s before restart
readiness	5 s × 2	**10 s** before traffic stops

Readiness reacts three times faster than liveness, deliberately. Taking a pod out of rotation is cheap and reversible; restarting it is expensive and destroys in-flight work. When in doubt, stop sending traffic — do not restart.

5. Internal architecture

The kubelet runs every probe locally — it never goes through a Service. Probe results feed two paths:

- **Liveness failure** → kubelet restarts the container in place. The pod object, its IP and its node do not change; only `restartCount` increments.

- **Readiness failure** → kubelet updates pod `status.conditions`. The endpoint controller observes it and removes the address from the EndpointSlice. kube-proxy on every node reprograms.

Only `Ready` pods appear in a Service's endpoints. That is the direct link between a probe and traffic routing, and the mechanism behind zero-downtime deployment.

6. Component interactions

```
kubelet   -> probe fails
kubelet   -> writes pod status Ready=false
endpoint controller -> removes address from EndpointSlice
kube-proxy (every node) -> reprograms iptables/IPVS
                        ~10-15s total, including propagation
```

That propagation delay is real, and it is why `preStop` exists.

7. Enterprise example

A payments platform requires every service to expose `/healthz` (process only) and `/readyz` (dependency-aware) before it can be deployed. The rule is enforced by an admission policy. It exists because a single team once pointed liveness at a database health check and turned a 40-second failover into a 20-minute platform outage.

8. Real-world analogy

A shop assistant. **Liveness** asks "are you conscious?" — if not, send them home. **Readiness** asks "can you serve the next customer right now?" — if they are on the phone to a supplier, stop queueing customers at them, but do not fire them.

Firing every assistant because the supplier's phone line is busy is the cascading-failure bug.

9. Best practices

Practice	Reason
Liveness checks the process only	Anything else causes correlated restarts
Readiness checks critical dependencies	That is the entire point
Always add a startup probe to a slow starter	Otherwise liveness races initialisation
Readiness faster than liveness	Stopping traffic is cheaper than restarting
`successThreshold: 1` on readiness	Anything higher delays recovery for no benefit
Classify dependencies critical vs non-critical	Not everything should take a pod out of rotation
Keep probes cheap	The kubelet runs them constantly, per pod
Exclude probe paths from access logs	Otherwise they dominate log volume — see `SILENT_PATHS`

10. Common mistakes

Mistake	Symptom
Liveness checks a dependency	Every replica restarts together on a blip → total outage
No readiness probe	Rollout "succeeds" while dropping traffic
No startup probe on a slow starter	CrashLoopBackOff on a cold cache
Liveness too aggressive	Restarts under load, never recovers
Readiness `successThreshold` > 1	Pod never becomes ready
Probe healthier than reality	Probes pass, payments fail
Expensive probe	Probe traffic becomes a measurable load

11. Security considerations

Threat	Control	Residual risk
Probe endpoint leaks internal state	Return status only, never config or versions of dependencies	Timing differences can still leak a little
Probe endpoint used as an unauthenticated DoS vector	Keep it cheap; do not let it trigger expensive work	Reachable from anywhere in the cluster until Day 4
Probe path reachable externally	Never expose `/healthz` through the Ingress	—

12. Performance considerations

- With 34 pods × 3 probes, the kubelet generates roughly **1,000 probe requests per minute** cluster-wide. Keep each one cheap.
- A readiness probe that performs a real database query multiplies that load onto the database.
- `SILENT_PATHS` in `axispay_common/metrics.py` excludes probe paths from access logging. Without it, probe traffic would be the single largest contributor to Loki storage on Day 5, burying real traffic.
- `timeoutSeconds` must exceed the worst-case response, or you get spurious failures under load — precisely when you least want them.

13. High availability

Probes are what make replica count meaningful. Three replicas where two cannot serve is one replica's worth of availability — and without readiness, Kubernetes cannot tell the difference. Readiness is also what makes rolling updates, node drains and PodDisruptionBudgets work correctly.

14. Disaster recovery

During a dependency outage, correct readiness probes mean pods leave rotation and rejoin automatically when the dependency returns — **no human action at all**. Incorrect liveness probes mean a thundering herd of restarts that extends the outage well beyond the original fault.

15. Monitoring

Metric	Threshold
`kube_pod_status_ready{condition="false"}`	Sustained > 2 min
`kube_pod_container_status_restarts_total` rate	Any sustained increase
`prober_probe_total{result="failed"}`	Rising
Ready replicas vs desired	Gap > 5 min — **the single best deployment alert**

16. Troubleshooting

Symptom	Cause	Command	Fix
All replicas restart together	**Liveness checks a dependency**	`kubectl get deploy -o yaml | grep -A3 livenessProbe`	Point liveness at `/healthz`
Pod never Ready	Readiness dependency unavailable	`kubectl exec … -- wget -qO- localhost:8080/readyz`	Fix the dependency
CrashLoop on startup	Liveness racing a slow start	`kubectl describe pod` → probe failed	Add/extend `startupProbe`
`connection refused` **on probe**	Wrong port, or not listening yet	`kubectl exec … -- ss -ltn`	Fix the port; add startup probe
Probes pass, traffic fails	Probe healthier than reality	Test the business endpoint	Make `/readyz` meaningful
Rollout hangs	New pods never Ready	`kubectl describe pod <new>`	Read the probe failure — the rollout is correctly refusing

Interview questions

1. **What is the difference between liveness and readiness?** *Consequence. Liveness failure restarts the container; readiness failure removes it from Service endpoints without restarting. Liveness must never check dependencies; readiness should.*
2. **Why must liveness never check a database?** *Because a brief database outage would fail liveness on every replica simultaneously, restarting the entire service. Restarting does not fix the database — it discards warm processes and connection pools and turns a dependency blip into a total outage.*
3. **What does a startup probe do that the others cannot?** *It suspends liveness entirely until it passes, giving a slow-starting container a generous budget without weakening liveness for the rest of its life.*
4. **How long after a pod becomes unready does traffic actually stop?** *(senior)* `periodSeconds × failureThreshold` *(10 s here) plus endpoint propagation to every node's kube-proxy — call it 10–15 s. That gap is why* `preStop` *exists: the pod must stay up long enough for the removal to reach every node.*
5. **Design readiness for a service with one critical and one optional dependency.** *(senior) Fail readiness only on the critical one; report the optional one as degraded but stay ready. In AxisPay,* `merchant-service` *is critical*

to `payment-service` (no pricing, no payment) but non-critical to `edge-gateway` (only `/account` degrades). That classification is a design decision written in code, not a default.

2.4 Horizontal Pod Autoscaling

1. What it is

A controller that adjusts a Deployment's replica count based on observed metrics.

2. Why it exists

Traffic is not constant. Manual scaling means someone is awake at 06:00 on Black Friday — and, historically, nobody scales back down.

3. The business problem

AxisPay's merchants expect six times normal volume on Black Friday. Last year the team scaled up manually and forgot to scale down for nine days.

4. How it works

```
desiredReplicas = ceil( currentReplicas × currentUtilisation ÷ targetUtilisation )

utilisation = actual usage ÷ the REQUEST
```

The request is the denominator. No request means no denominator: the HPA reports `<unknown>` and does nothing, forever, with no error and no event. This is the arithmetic dependency that makes resources a prerequisite for autoscaling.

Worked example: 4 pods at 140% of request, target 70% → `ceil(4 × 140 / 70)` = **8 replicas**.

5. Internal architecture

The HPA controller reconciles every 15 seconds:

1. Fetch metrics from the metrics API (served by metrics-server, which scrapes each kubelet's cAdvisor).
2. Compute utilisation per pod as a percentage of its request.
3. Average across ready pods; apply the formula.
4. Apply `behavior` policies and stabilisation windows.
5. Write `spec.replicas` on the Deployment.

The HPA owns `spec.replicas`. Leaving `replicas` in a manifest that also has an HPA causes them to fight on every `apply`.

6. Component interactions

```
pods -> kubelet/cAdvisor -> metrics-server -> HPA -> Deployment -> ReplicaSet -> pods
```

A closed loop. If metrics-server is down, the whole loop stops silently.

7. Enterprise example

A payments platform scales its authorisation service on CPU and its notification worker on **queue depth** via a custom metric. The distinction matters: authorisation is CPU-bound, but a notification worker is bound by backlog, and CPU tells you nothing about it.

8. Real-world analogy

A supermarket opening more tills as queues lengthen. Tills open quickly when queues form, and close slowly afterwards — because opening and closing repeatedly is worse than leaving one open a few minutes longer.

Where it breaks: a supermarket manager can see the queue. The HPA sees only CPU. If tills are slow because the card terminal is broken, opening more tills does nothing.

9. Best practices

Practice	Reason
Never deploy an HPA without resource requests	No denominator, no autoscaling, no error
Scale up fast, scale down slowly	Prevents flapping
`minReplicas` ≥ 3 for critical services	Survives a node loss even at minimum
Size the ResourceQuota for `maxReplicas`	Or scaling stalls during the spike
Remove `replicas` from HPA-managed manifests	Otherwise HPA and `apply` fight
Scale on the metric that actually saturates	CPU is often the wrong one
Alert when an HPA sits at `maxReplicas`	You have run out of headroom

10. Common mistakes

Mistake	Symptom
No CPU request	`TARGETS: <unknown>`, HPA does nothing, no error
Quota sized for steady state	Scaling stops mid-spike with `FailedCreate`
Scale-down too aggressive	Flapping: scale down → latency rises → scale up
`replicas` left in the manifest	Replica count oscillates on every apply
Expecting HPA to fix a broken dependency	CPU *falls* when pods cannot serve; HPA does nothing
Scaling a stateful workload on CPU	New replicas do not share the state — see the velocity bug below

11. Security considerations

Threat	Control	Residual risk
Traffic flood drives cost-amplification scaling	`maxReplicas`, quota, rate limiting at the edge	An attacker can still drive you to max
HPA scaling a compromised workload wider	Standard workload controls	HPA has no notion of correctness
metrics-server as an attack surface	RBAC; it is cluster-privileged	Compromise yields cluster-wide metrics

12. Performance considerations

- **15-second reconcile interval** plus metrics-server's scrape interval means the HPA reacts on the order of 30–60 seconds. It cannot absorb a sub-minute spike; that is what headroom in `minReplicas` is for.
- **Scale-down stabilisation (300 s)** deliberately trades cost for stability.
- **Pod startup time** is the real limit on how fast you can scale. If a pod takes 45 seconds to become ready, the HPA cannot help you inside that window.

> ### The in-memory state bug — worth understanding properly
>
> `fraud-service` keeps velocity counters in memory. With 6 replicas behind a Service, each pod sees roughly one sixth of the traffic. A rule of "more than 8 attempts in 5 minutes" now effectively fires at **48** attempts.
>
> **The fraud control silently weakens as you scale.** Nothing errors. No alert fires. Scaling up for performance quietly degrades a security control.
>
> This is why per-replica state and horizontal scaling are fundamentally in tension, and it is exactly what Redis fixes on Day 3.

13. High availability

`minReplicas` is the real HA lever — set it so that losing a node still leaves enough capacity. AxisPay uses `minReplicas: 3` for `payment-service` (one per node) so a node failure never drops below two.

14. Disaster recovery

HPAs are configuration and recover with the manifest. During an incident they can work against you: a service failing slowly may consume *less* CPU, causing the HPA to scale *down* just as demand rises. Knowing when to suspend autoscaling is part of incident response.

15. Monitoring

Metric	Threshold
`kube_horizontalpodautoscaler_status_current_replicas` == `_spec_max_replicas`	Sustained — out of headroom
`kube_horizontalpodautoscaler_status_condition{condition="ScalingActive",status="false"}`	Any — usually `<unknown>` metrics
Scale events per hour	High = flapping
metrics-server availability	Any gap disables all autoscaling

16. Troubleshooting

Symptom	Cause	Command	Fix
`TARGETS: <unknown>`	No CPU request, or metrics-server down	`kubectl describe hpa` ; `kubectl top pods`	Add requests; check metrics-server
Never scales up	Utilisation below target	`kubectl describe hpa`	More load, or lower the target
Scales up then straight down	Stabilisation too short	`kubectl get hpa -o yaml`	Lengthen `scaleDown` window
Stops below `maxReplicas`	Quota reached	`kubectl describe rs` ; `describe quota`	Size the quota for max
Fights your `apply`	`replicas` still in the manifest	—	Remove it
`FailedGetResourceMetric`	metrics-server unhealthy	`kubectl -n kube-system logs -l k8s-app=metrics-server`	Restart; wait for a scrape window

Interview questions

1. **Why does an HPA need resource requests?** *Utilisation is a percentage of the request. With no request there is no denominator, so the HPA reports* `<unknown>` *and does nothing — silently.*
2. **Give the formula and work an example.** `desired = ceil(current × utilisation / target)`. *Four pods at 140% against a 70% target gives* `ceil(8)` *= 8 replicas.*
3. **Why is scale-down slower than scale-up?** *To prevent flapping. Removing capacity raises latency, which triggers scale-up, which triggers scale-down again. Each cycle costs a pod start and a cold cache. Five minutes of extra capacity is far cheaper than the oscillation.*

4. **A dependency fails and payments start erroring. What does the HPA do?** *(senior) Nothing, or it scales down — because CPU usage falls when pods cannot serve. Autoscaling responds to load, not correctness. This is the clearest illustration of its boundary.*
5. **Your service keeps per-user rate-limit counters in memory and is behind an HPA. What breaks?** *(senior) Each replica sees a fraction of the traffic, so the effective limit is multiplied by the replica count. The control silently weakens as you scale, with no error. The fix is shared state — Redis — or consistent hashing so a given key always reaches the same replica.*

2.5 DaemonSet, Job and CronJob

1. What it is

Three controllers for workloads a Deployment cannot express: one per node, run-to-completion, and run-on-a-schedule.

2. Why it exists

Not everything is N interchangeable replicas running forever.

3. The business problem

AxisPay needs PCI file-integrity monitoring on **every** node including new ones, a reconciliation run that compares the ledger against the acquirer position and then stops, and settlement at 23:00 **exactly once**.

4. How it works

Controller	Count decided by	Complete when
Deployment	You (`replicas`)	Never — runs forever
DaemonSet	**The node inventory** — no `replicas` field	Never
Job	You (`completions` , `parallelism`)	**Pod exits 0**
CronJob	The schedule	Each Job completes

5. Internal architecture

DaemonSet. The controller watches Nodes, not a replica count. Add a node and a pod appears; drain a node and it goes. Because agents usually must run everywhere — including tainted control-plane nodes — a DaemonSet commonly carries tolerations that ordinary workloads do not.

Job. `backoffLimit` bounds pod retries with exponential back-off. `activeDeadlineSeconds` is a hard wall-clock ceiling. `restartPolicy` must be `Never` or `OnFailure` — `Always` is rejected, because a container that restarts on success could never complete.

CronJob. Creates Jobs on a schedule. Key fields:

Field	AxisPay value	Reason
`timeZone`	`Africa/Johannesburg`	The cluster runs UTC; the merchant's end-of-day does not
`concurrencyPolicy`	`Forbid`	Settlement must never double-run
`startingDeadlineSeconds`	600	Run late, but not absurdly late
`successfulJobsHistoryLimit`	3	Enough to debug; not enough to fill etcd

> **`timeZone` is not cosmetic.** `0 23 * * *` with no `timeZone` on a UTC cluster runs at **01:00 the next day** in Johannesburg. The Tuesday batch runs on Wednesday and may include Wednesday's early transactions. That is an accounting defect, and it is completely silent.

6. Component interactions

```
DaemonSet controller -> watches Nodes -> creates one pod per eligible node
Job controller       -> creates pods -> counts successes -> marks complete/failed
CronJob controller   -> evaluates schedule -> creates a Job -> applies concurrencyPolicy
```

7. Enterprise example

A bank runs four DaemonSets on every node — log shipper, metrics agent, file-integrity monitor, vulnerability scanner — all tolerating every taint, because an unmonitored node is a compliance finding. Settlement runs as a CronJob with `Forbid`, and the application is *also* idempotent, because a Job pod can be killed after doing its work but before recording it.

8. Real-world analogy

DaemonSet: a fire extinguisher on every floor. You do not order "three extinguishers" — you order one per floor, and a new floor gets one automatically. **Job:** the annual stocktake. It starts, finishes, and produces a report. **CronJob:** the cleaner who comes at 23:00 every night.

Where it breaks: if the cleaner is still working at 23:00 tomorrow, `Forbid` means tomorrow's visit is skipped entirely — not queued. For settlement that is correct; for some workloads it is not.

9. Best practices

Practice	Reason
DaemonSet resources tightly capped	An agent that starves its host is worse than no agent
Always set `backoffLimit` and `activeDeadlineSeconds`	Bound both retries and wall-clock time
`ttlSecondsAfterFinished` on Jobs	Otherwise completed Jobs accumulate in etcd
Always set `timeZone` on a CronJob	The cluster runs UTC
`concurrencyPolicy: Forbid` for money movement	Two settlement runs would double-count
Make the Job idempotent anyway	Kubernetes gives at-most-once *scheduling*, not exactly-once *execution*
`updateStrategy.maxUnavailable: 1` on DaemonSets	Update one node at a time

10. Common mistakes

Mistake	Symptom
`restartPolicy: Always` on a Job	Rejected at admission
No `activeDeadlineSeconds`	A hung nightly job runs until someone notices in the morning
No `timeZone`	Batch runs on the wrong calendar day, silently
`concurrencyPolicy: Allow` for settlement	Double-counted money
DaemonSet with no tolerations	Missing on tainted nodes — the ones you most need to watch
Treating Jobs as exactly-once	A pod killed after work but before recording repeats the work
No history limits	Thousands of completed Jobs in etcd

11. Security considerations

Threat	Control	Residual risk
DaemonSets often need host access	Minimise `hostPath`; drop capabilities; read-only mounts	Node-level agents are inherently privileged
A compromised DaemonSet runs on every node	Strict RBAC and image provenance	Blast radius is the whole cluster by design
Jobs handling sensitive data leave logs	Short `ttlSecondsAfterFinished`; do not log payloads	Logs may already be shipped
CronJob schedule tampering	RBAC on CronJob objects	A schedule change is easy to miss in review

12. Performance considerations

- A DaemonSet multiplies its resource footprint by the node count. 50m × 200 nodes is 10 CPUs of agent.
- `parallelism` and `completions` on a Job control throughput; `completionMode: Indexed` lets each pod take a deterministic slice.
- CronJobs firing at the same minute across many namespaces cause a thundering herd — stagger them.

13. High availability

DaemonSets are inherently distributed — one per node. Jobs are not HA: if the node running a Job pod fails, the pod is recreated elsewhere and the work restarts, which is exactly why idempotency matters. CronJob scheduling depends on the controller-manager; a missed window is governed by `startingDeadlineSeconds`.

14. Disaster recovery

For batch workloads, the recovery question is **"did it run, and did it complete?"** — not "is it up?". Keep a durable record of completion outside Kubernetes (a row in the database), because Job history is pruned. AxisPay's settlement writes its own record for exactly this reason.

15. Monitoring

Metric	Threshold
`kube_daemonset_status_number_unavailable`	> 0
`kube_daemonset_status_desired_number_scheduled` vs node count	Mismatch = a taint you do not tolerate
`kube_job_status_failed`	Any
`kube_cronjob_status_last_schedule_time`	Older than the schedule interval — **it did not fire**
Job duration vs historical	Sudden increase

16. Troubleshooting

Symptom	Cause	Command	Fix
DaemonSet missing on a node	Taint not tolerated	`kubectl describe node <n>` → Taints	Add a toleration
Job retries forever	No `backoffLimit`	`kubectl describe job`	Set it
Job `Error`, pod exits non-zero	Application failure	`kubectl logs job/<name>`	Fix the cause
`restartPolicy: Always` rejected	Jobs forbid it	—	`Never` or `OnFailure`
CronJob never fires	Bad schedule or `suspend: true`	`kubectl get cronjob -o yaml`	Fix it
CronJob fires at the wrong hour	No `timeZone`	Same	Set `timeZone`
Jobs accumulate	No TTL or history limits	`kubectl get jobs`	Set `ttlSecondsAfterFinished`

Interview questions

1. **Why does a DaemonSet have no `replicas` field?** *Because the count is determined by the node inventory, not by you. Add a node and a pod appears automatically. The equivalent of "scaling" is changing which nodes are eligible — via `nodeSelector` or tolerations.*
2. **What makes a Job complete?** *The pod exiting 0. That is why `restartPolicy: Always` is rejected — a container restarting on success could never finish.*
3. **Why must a CronJob set `timeZone`?** *The cluster runs UTC. `0 23 * * *` for a Johannesburg merchant fires at 01:00 the following local day, so the batch settles the wrong calendar day. It is silent and it is an accounting defect.*
4. **How would you guarantee a settlement runs exactly once?** *(senior) You cannot, from Kubernetes alone. `concurrencyPolicy: Forbid` gives at-most-once scheduling, but a Job pod can be killed after doing its work*

and before recording it, so the retry repeats the work. The guarantee must come from the application: an idempotency key or a uniquely-constrained completion record in the database, checked before starting.

5. **When would `concurrencyPolicy: Replace` be right, and when catastrophic?** *(senior) Right for an idempotent refresh where only the latest result matters — a cache warm, a report regeneration. Catastrophic for money movement: it kills a run mid-way, leaving partial work with no record of how far it got.*

2.6 Rolling updates and graceful shutdown

1. What it is

Replacing every pod of a Deployment with a new version, one controlled step at a time, and terminating the old ones without severing work in flight.

2. Why it exists

Stopping everything and starting the new version is an outage. For a platform that earns per transaction, an outage is revenue that does not arrive.

3. The business problem

Change request CR-2026-0812: `payment-service` v1.0.0 → v1.1.0, during trading, adding fraud scoring and acquirer routing. **No merchant may see a single failed authorisation.**

4. How it works

The Deployment creates a **new ReplicaSet** and shifts replicas across:

Setting	Value	Meaning
`maxUnavailable`	**0**	Never fewer than 3 **ready** pods — capacity never drops
`maxSurge`	1	At most 4 pods exist at once

With `maxUnavailable: 0` the rollout is strictly **add-then-remove**. It needs headroom for one extra pod and it is slower. On a payment path that is the correct trade.

Readiness is the gate. A new pod counts as available only when its readiness probe passes. Remove the probe and "ready" degrades to "the process started" — and the controller removes an old pod before the new one can serve.

5. Internal architecture

```
t0   RS-A: 3 ready (v1.0.0)                    total 3, serving 3
t1   RS-B: +1 pod created, not ready           total 4, serving 3
t2   new pod passes READINESS, joins endpoints total 4, serving 4
t3   one old pod terminates (preStop, drain)   total 3, serving 3
...  repeat
t5   RS-B: 3 ready; RS-A: 0 replicas, RETAINED total 3, serving 3
```

`progressDeadlineSeconds: 300` marks the Deployment failed if it has not progressed — without it a broken release sits "in progress" forever and no pipeline ever fails.

6. Component interactions — termination

Two things happen **in parallel** when a pod is deleted:

#	Event	Why it matters
1	Endpoint controller removes the address; kube-proxy reprograms on every node	Eventually consistent — it takes time to propagate
2	`preStop` runs (`sleep 8`)	The pause that lets step 1 reach every node
3	**SIGTERM** to PID 1	App marks itself unready and finishes in-flight work
4	Up to `terminationGracePeriodSeconds` (45 s)	Must exceed the **longest** in-flight operation
5	**SIGKILL**	Cannot be caught. Anything still running is severed.

> **The `ENTRYPOINT` form matters.** With the shell form, PID 1 is `/bin/sh`, which does **not** forward SIGTERM. Kubernetes waits the full grace period and then SIGKILLs every pod on every rollout — turning zero-downtime into guaranteed-downtime, with no error anywhere. Every AxisPay Dockerfile uses the exec form for this reason.

7. Enterprise example

A payments platform deploys 40 times a day with `maxUnavailable: 0`, a PodDisruptionBudget on every workload, and anti-affinity across zones. A failed readiness probe halts the rollout automatically and the previous version keeps serving. The release fails safe, without a human deciding anything.

8. Real-world analogy

Replacing staff mid-shift without leaving the counter unstaffed: bring the new person in, wait until they are actually ready to serve, then let one of the old team finish their current customer and leave. Repeat.

Where it breaks: a human can say "give me two minutes to finish". A container gets SIGTERM and a fixed grace period, then SIGKILL regardless.

9. Best practices

Practice	Reason
`maxUnavailable: 0` for user-facing services	Capacity never drops
Always define a readiness probe	Without it the rollout is not zero-downtime, whatever the strategy says
Set `progressDeadlineSeconds`	A stuck rollout must eventually fail
`preStop` pause before SIGTERM	Lets endpoint removal propagate
Grace period > longest in-flight operation	Not the average
Exec-form `ENTRYPOINT`	Or SIGTERM never reaches your process
Record `kubernetes.io/change-cause`	`rollout history` becomes readable in an incident
Measure the rollout	A release you did not measure proves nothing

10. Common mistakes

Mistake	Symptom
No readiness probe	Traffic dropped during every rollout
`maxUnavailable > 0` on a critical path	Capacity dips under load
Shell-form `ENTRYPOINT`	Graceful shutdown silently never happens
Grace period too short	In-flight payments severed by SIGKILL
No `preStop`	502s at the moment of termination
Changing only `spec.replicas` and expecting a rollout	Only `spec.template` changes create a ReplicaSet
Relying on `rollout undo` beyond `revisionHistoryLimit`	Nothing to roll back to

11. Security considerations

Threat	Control	Residual risk
Rollback re-introduces a vulnerable image	Scan on pull; block bad digests at admission	Emergency rollback may bypass policy
Anyone with `update` on Deployments runs arbitrary images	RBAC; registry allow-listing at admission	CI credentials remain powerful
In-flight data lost on abrupt termination	Grace period + `preStop` + idempotency	SIGKILL is always possible

12. Performance considerations

- Rollout duration ≈ `replicas × (pod startup + readiness delay) ÷ maxSurge`. A high `initialDelaySeconds` makes every release slow.
- `maxUnavailable: 0` requires the cluster to fit `replicas + maxSurge` — real capacity, not theoretical.
- With 200 replicas, use a **percentage** rather than an absolute surge.
- Old ReplicaSets consume etcd and API server memory. Keep `revisionHistoryLimit` modest.

13. High availability

Rolling updates are only zero-downtime when readiness probes, `maxUnavailable: 0`, sufficient capacity **and** graceful shutdown are all in place. Missing any one of them silently degrades the guarantee — and the failure appears as sporadic 502s during deploys, which is easy to dismiss.

14. Disaster recovery

`kubectl rollout undo` is near-instant because the old ReplicaSet is retained at zero replicas — no image pull, no new object. Beyond `revisionHistoryLimit`, recovery means re-applying an older manifest from Git.

Rollback is only safe if the change is backwards-compatible. A release that migrated a database schema cannot be rolled back by scaling a ReplicaSet — which is why the Day 5 capstone separates the schema migration from the application rollout.

15. Monitoring

Metric	Threshold
`kube_deployment_status_replicas_available` vs `_spec_replicas`	Gap > 5 min
`kube_deployment_status_observed_generation` vs `metadata.generation`	Lagging = controller not reconciling
Error rate during a deploy window	Any increase — this is what loadgen measures
Two ReplicaSets non-zero for a long period	Stuck rollout
Pod termination duration	Approaching the grace period

16. Troubleshooting

Symptom	Cause	Command	Fix
Rollout stuck part-way	New pods never Ready	`kubectl describe pod <new>`	Read the probe failure
`ProgressDeadlineExceeded`	Exceeded the deadline	`kubectl describe deploy`	Investigate, then `rollout undo`
Errors during rollout	No readiness probe or `maxUnavailable > 0`	`kubectl get deploy -o yaml`	Add readiness; set 0
Rollout does not start	Nothing in `spec.template` changed	`kubectl rollout history`	Change the template
Pods take the full grace period to die	SIGTERM not reaching the process	Check `ENTRYPOINT` form	Use exec form
502s at termination	Missing/short `preStop`	Check `lifecycle.preStop`	Add or extend the pause
Cannot roll back	History pruned	`kubectl rollout history`	Re-apply from Git

Interview questions

1. **What makes a rolling update zero-downtime?** *A readiness probe plus `maxUnavailable: 0`. The probe means "ready" reflects real serving capability; `maxUnavailable: 0` means no old pod is removed until a new one is genuinely ready.*
2. **`replicas: 3`, `maxSurge: 1`, `maxUnavailable: 0`. Maximum pods, minimum serving?** *Maximum 4, minimum serving 3. With `maxSurge: 0` as well, the rollout could never start — Kubernetes rejects that combination.*

3. **Describe the termination sequence.** *Pod marked Terminating and endpoint removal begins in parallel; `preStop` runs; SIGTERM to PID 1; up to `terminationGracePeriodSeconds` to exit; then SIGKILL. `preStop` exists because endpoint removal is eventually consistent across nodes.*
4. **Why does the `ENTRYPOINT` form affect graceful shutdown?** *(senior) Shell form makes `/bin/sh` PID 1, and it does not forward SIGTERM to the application. The process never learns it should drain, Kubernetes waits the full grace period, then SIGKILLs. Every rollout silently severs in-flight work, with no error anywhere.*
5. **When is `Recreate` the correct strategy?** *(senior) When two versions cannot run simultaneously — a schema migration that is not backwards-compatible, or a singleton holding an exclusive lock. It is a deliberate outage in exchange for correctness, and it should be a conscious decision rather than a default.*

Day 2 cheat sheet

Resources

```
kubectl top nodes
kubectl top pods -n axispay-core --containers

# what is declared vs what QoS resulted
kubectl get pods -n axispay-core -o custom-columns=\
NAME:.metadata.name,\
CPU_REQ:.spec.containers[0].resources.requests.cpu,\
CPU_LIM:.spec.containers[0].resources.limits.cpu,\
QOS:.status.qosClass

# THE command for invisible latency — CPU throttling
kubectl exec -n axispay-core <pod> -- cat /sys/fs/cgroup/cpu.stat
#   nr_throttled / throttled_usec climbing = the limit is too low

kubectl describe quota  -n axispay-core
kubectl describe limitrange -n axispay-core
```

Probes

Probe	Path	Fails →	Checks dependencies
startup	`/startupz`	keep waiting, liveness suspended	No
liveness	`/healthz`	**RESTART**	**Never**
readiness	`/readyz`	**leave endpoints**	**Yes**

```
kubectl get deploy <d> -n <ns> -o yaml | grep -A5 Probe
kubectl describe pod <pod> | grep -A3 "Liveness\|Readiness"
kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>
```

Autoscaling

```
desired = ceil( current × utilisation ÷ target )      utilisation = usage ÷ REQUEST
```

```
kubectl get hpa -n axispay-core          # TARGETS <unknown> = no request
kubectl describe hpa <name> -n axispay-core | tail -15
```

Workload types

Need	Controller
N copies always running	Deployment
One per node	**DaemonSet** (no `replicas` field)
Run once and stop	**Job** (`restartPolicy` Never/OnFailure)
Run on a schedule	**CronJob** (set `timeZone` !)

```
kubectl get daemonset,job,cronjob -A
kubectl logs -n axispay-async job/recon-worker
kubectl create job --from=cronjob/settlement-cron manual-run -n axispay-async
```

Rollouts

```
kubectl rollout status  deployment/<d> -n <ns>
kubectl rollout history deployment/<d> -n <ns>
kubectl rollout undo    deployment/<d> -n <ns>
kubectl rollout undo    deployment/<d> -n <ns> --to-revision=3

# measure it
kubectl port-forward -n axispay-ops deploy/loadgen 8090:8080 &
curl -s -X POST localhost:8090/api/v1/loadgen/start -H 'Content-Type: application/json' -d
'{"rps":40}'
curl -s localhost:8090/api/v1/loadgen/stats | python3 -m json.tool
```

The two failure symptoms

	CPU limit exceeded	Memory limit exceeded
Result	**Throttled**	**OOMKilled**
Exit code	— (still running)	**137**
Restart	No	Yes
Log line	**None**	None from the app — SIGKILL cannot be caught
Event	**None**	Yes
Find it with	`cat /sys/fs/cgroup/cpu.stat`	`kubectl describe pod` → Last State

Day 2 review questions

1. Which does the scheduler use — requests or limits? Which does the kernel enforce?
2. What happens when a container exceeds its CPU limit? Its memory limit?
3. What are the three QoS classes, and in what order are they evicted?
4. Why does a ResourceQuota need a LimitRange?
5. Where does a quota rejection appear, and why is that easy to miss?
6. Name the three probes and, for each, the consequence of failure.
7. Why must a liveness probe never check a database?
8. How long after a pod becomes unready does traffic actually stop, and why is it not instant?
9. Give the HPA formula. What is the denominator?
10. An HPA shows `TARGETS: <unknown>`. What is wrong?
11. Why is scale-down slower than scale-up?
12. A dependency fails and payments error. What does the HPA do, and why?
13. Why does a DaemonSet have no `replicas` field?
14. What makes a Job complete? Why is `restartPolicy: Always` rejected?
15. Why must a CronJob set `timeZone`?
16. `replicas: 3`, `maxSurge: 1`, `maxUnavailable: 0` — max pods, min serving?
17. Describe the termination sequence from delete to SIGKILL.
18. Why does the Dockerfile `ENTRYPOINT` form affect graceful shutdown?
19. What are the two things that together make a rolling update zero-downtime?
20. `requests.memory: 96Mi`, `limits.memory: 48Mi`. What happens, and what should have caught it?

Answers: `documents/assessments/answer-keys/day2-answer-key.md`

Day 2 summary

You built: resource requests and limits on every workload · namespace governance verified to permit full autoscaling · three probes correctly separated by consequence · two HPAs scaling on real CPU load · a DaemonSet, a Job and a CronJob · a zero-downtime release of v1.1.0 under 40 rps with **zero failed payments**.

You proved: CPU throttling breaks an SLO with no log line · a wrong liveness probe turns a 40-second blip into a total outage · an HPA without requests does nothing and says nothing · removing the readiness probe drops real payments.

What is still missing:

Gap	Consequence	Fixed
No database	Every payment lost on restart. Nothing to reconcile.	L3.5
Config in manifests	A log-level change redeploys every service	L3.1
JWT key in a plain env var	Visible to anyone with namespace read access	L3.2
Fraud counters in memory	The control weakens as you scale — you found this yourself	L3.5
No persistent storage	A StatefulSet has nowhere to keep data	L3.3, L3.4
Containers run as root	A container escape becomes a host compromise	L3.7

Tonight (optional, 15 minutes): delete a `payment-service` pod, then try to fetch a payment you created this morning. Understanding *why* it is gone is the whole of Day 3.